*PROJECT REPORT*

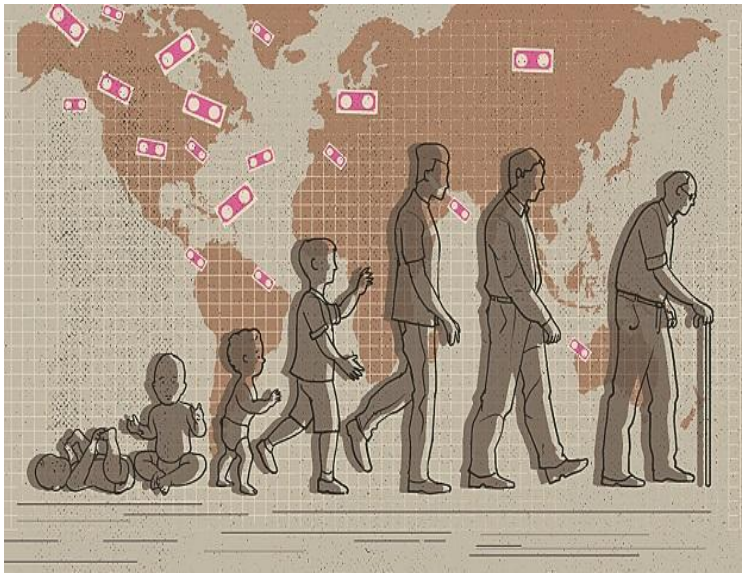
# A COMPREHENSIVE MEASURE OF WELL-BEING

## *HDI Insight : AI-Powered Human Development Index Prediction Platform*

Prepared in partial fulfillment of the Project Development Lifecycle

AT

# APSCHE




**Submitted By:**

**Ratna Sri Dayam**

**Kovvuri Harshitha**

**Maram Jhansi**

**Lova Veerraju Panneru**

**Vamsi Krishna Sinisetti**

## Project Description:

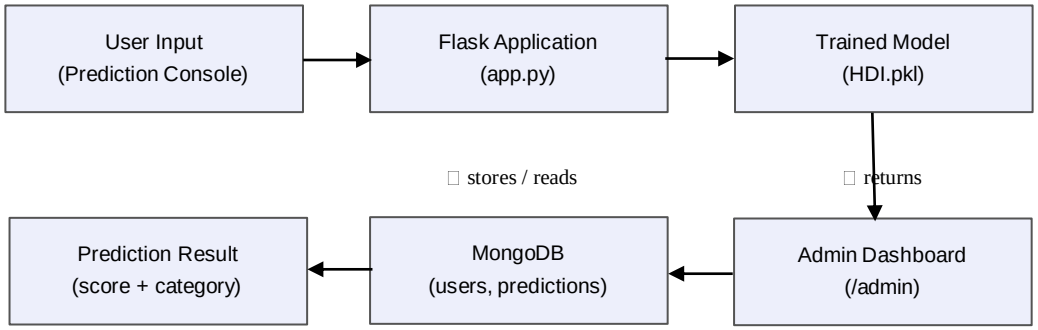
HDI Insight harnesses machine learning to provide an intelligent, data-driven estimate of a country's Human Development Index (HDI), offering users a fast and accessible way to explore how key socio-economic indicators translate into overall human development. The platform includes a Prediction Console, which accepts Country, Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users (%) as inputs, and a Result view, which returns the predicted HDI score on a visual gauge together with a four-tier development classification (Low, Medium, High, or Very High HDI).

Utilizing a Linear Regression model trained with scikit-learn on a 195-country Kaggle dataset, HDI Insight processes user inputs to deliver an instant, statistically grounded HDI estimate, achieving an R-squared score of 0.955 on a 90/10 train-test split. Built with Flask and backed by MongoDB for secure user registration, login, and prediction logging, the platform ensures a seamless, authenticated experience. With the trained model serialized via Pickle and an administrator dashboard for usage tracking, HDI Insight empowers students, researchers, and policy enthusiasts to explore human development data with confidence.

## Scenarios

- **Scenario 1:** A new user visits HDI Insight, registers with their name and email, and logs in. From the home page, they review the model's headline statistics (R-squared 0.955, 195 countries modeled, 90/10 split) and the four HDI dimensions, then proceed to the Prediction Console.
- **Scenario 2:** A student researching a specific country selects it from the dropdown and enters its Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users (%). HDI Insight returns a Medium HDI classification with a score of 0.63, displayed on a visual gauge with a 'Run Another Prediction' option.
- **Scenario 3:** A course administrator logs in with an administrator role and opens the Admin Dashboard to review total registered users, total predictions made, active ML models, and a live log of recent predictions, including any submissions that fell outside the documented valid indicator ranges.

## Technical Architecture:




**Description:** HDI Insight utilizes a modular three-tier architecture to deliver authenticated, ML-driven predictions. The frontend provides a responsive, gold-accented interface built with HTML and CSS. The Flask-based backend handles routing, input validation, and classification logic, and loads the serialized regression model (HDI.pkl) at startup. MongoDB serves as the persistence layer for user accounts and the prediction activity log surfaced on the administrator dashboard.

**Note:** Because this project uses MongoDB for authentication and prediction logging, a conceptual Entity-Relationship structure is included below: a **User's** collection (name, email, password, role, created_at) and a **Predictions** collection (timestamp, user_name, user_email, score, category), related one-to-many from Users to Predictions via user_email.

## Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation (flask.palletsprojects.com)
- **scikit-learn Familiarity:** scikit-learn Documentation (scikit-learn.org/stable)
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation (docs.python.org)
- **MongoDB and PyMongo Familiarity:** MongoDB Documentation (mongodb.com/docs)
- **Version Control with Git:** Git Documentation (git-scm.com/doc)
- **Development Environment Setup:** Flask Installation Guide
- **Pandas / NumPy / Pickle:** Pandas, NumPy, and Python Pickle Module Documentation

## Project Workflow:

### Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Source the HDI dataset from Kaggle and load it for exploration.
- **Activity 1.2:** Research and select the appropriate regression model for HDI prediction.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, MongoDB, and the trained model.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, scikit-learn, and MongoDB.

### Milestone 2: Core Functionalities Development

- **Activity 2.1:** Clean the dataset and select dependent/independent variables.
- **Activity 2.2:** Split the data and train the Linear Regression model.
- **Activity 2.3:** Evaluate the model and serialize it with Pickle.

### Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for authentication, prediction, and the admin dashboard.
- **Activity 3.2:** Integrate MongoDB for user accounts and prediction logging.

### Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the registration and login pages.
- **Activity 4.2:** Design the home landing page with model statistics and HDI methodology.
- **Activity 4.3:** Design the Prediction Console and result gauge page.
- **Activity 4.4:** Design the Admin Dashboard.

### Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment, configuring MongoDB and ensuring all dependencies are installed.
- **Activity 5.2:** Run and test the Flask application locally.

### Milestone 6: Conclusion

# Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate regression model and dataset for HDI Insight's prediction needs. This involves researching the relationship between candidate indicators and HDI, and ensuring the chosen model and feature set align with the application's goal of producing a reliable, interpretable HDI estimate.

## Activity 1.1: Source and Load the Dataset

The dataset was sourced from Kaggle and mirrored at github.com/Guided-Projects/HumanDevelopmentIndex/tree/main/Dataset. It contains 195 rows (one per country) and 82 columns of human development indicators.

```
import pandas as pd

data = pd.read_csv('Dataset/HDI.csv')
data.head()
print(data.shape) # (195, 82)
```

## Activity 1.2: Research and Select the Appropriate Model

- **Understand the Project Requirements:** Review the specific needs of HDI Insight, focusing on producing a continuous HDI score from a small set of easily obtainable indicators.
- **Explore Candidate Indicators:** Use a correlation heatmap to examine which of the 82 columns correlate most strongly with HDI.
- **Evaluate Model Options:** Compare regression approaches based on interpretability, training speed, and fit quality.
- **Select the Optimal Model:** Choose scikit-learn's **LinearRegression** for its interpretability and strong fit, trained on a 90/10 split and achieving an **R-squared of 0.955**.

## Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML/CSS), Flask backend, MongoDB, and the serialized model.
- **Detail Frontend Functionality:** Outline how users register, log in, review the home page, and submit indicator values on the Prediction Console.
- **Outline Backend Responsibilities:** Specify how Flask handles authentication routes, validates prediction form input, loads HDI.pkl, and classifies the resulting score.
- **Describe Data Integration Points:** Define how MongoDB stores users and predictions, and how the admin route aggregates that data for the dashboard.

## Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Core Libraries:** Flask, pandas, numpy, matplotlib, seaborn, scikit-learn, pymongo.

```
(myenv) D:\projects> pip install Flask==3.0.3
(myenv) D:\projects> pip install scikit-learn pandas numpy matplotlib seaborn pymongo
```

- **Install and Start MongoDB:** Install MongoDB Community Edition and confirm the local service is running.

```
mongod --version
mongod # starts the local MongoDB service
```

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files, the dataset, and the training notebook.

```
ML - 0027 - Human Development Index/
|-- Dataset/
| `-- HDI.csv
|-- Flask/
| |-- templates/
| | |-- home.html
| | |-- login.html
| | |-- register.html
| | |-- prediction.html
| | |-- result.html
| | `-- admin.html
| |-- static/
| |-- app.py
| `-- HDI.pkl
`-- Training/
    `-- HumDevIndex.ipynb
```

# Milestone 2: Core Functionalities Development

## Activity 2.1: Data Cleaning and Feature Selection

Identify and handle missing values, then select the independent variables (X) and dependent variable (Y) based on correlation analysis.

```
X = data.iloc[:, [5, 6, 7, 8]] # Life Expectancy, Schooling, GNI, Internet Users
Y = data.iloc[:, 4]              # HDI Score

X.isnull().sum()
X = X.fillna(X.mean())
```

## Activity 2.2: Split and Train the Model

Split the dataset using a 90/10 ratio, then fit a Linear Regression model on the training data.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.1, random_state=0)

reg = LinearRegression()
reg.fit(X_train, y_train)

# testing with few values
y_pred = reg.predict([[13, 72.0, 5.2, 3341.0, 14.4]])
print(y_pred) # [[0.59410218]]
```

## Activity 2.3: Evaluate the Model

```
from sklearn.metrics import r2_score

y_pred = reg.predict(X_test)
score = r2_score(y_test, y_pred)
print(score) # 0.955
```

The model achieves an R-squared score of **0.955** on the held-out test set, confirming a strong fit and supporting its use as the production model behind the Prediction Console.

## Activity 2.4: Serialize the Model with Pickle

```
import pickle

with open('HDI.pkl', 'wb') as f:
    pickle.dump(reg, f)
```

The resulting HDI.pkl file is placed inside the Flask/ directory so it can be loaded directly by the web application at startup, avoiding the need to retrain the model on every request.

## Milestone 3: App.py Development

Milestone 3 focuses on creating the core logic of app.py, which serves as the backbone of HDI Insight. This includes authentication routes, the prediction endpoint, classification logic, and the admin dashboard route, each backed by MongoDB where relevant.

### Activity 3.1: Writing the Main Application Logic

```
from flask import Flask, render_template, request, redirect, session
from pymongo import MongoClient
from datetime import datetime
import pickle
import numpy as np

app = Flask(__name__)
app.secret_key = 'hdi-insight-secret'
model = pickle.load(open('HDI.pkl', 'rb'))

client = MongoClient('mongodb://localhost:27017/')
db = client['hdi_insight']
users = db['users']
predictions = db['predictions']
```

**Home and Prediction Routes:**

```
@app.route('/')
def home():
    return render_template('home.html')

@app.route('/Prediction')
def prediction_form():
    return render_template('prediction.html')
```

**Predict Route:** Retrieves form input, converts it to numerical format, passes it to the trained model, derives the classification tier, logs the request to MongoDB, and renders the result page.

```
@app.route('/predict', methods=['POST'])
def predict():
    features = [float(request.form[f]) for f in
                ['life_expectancy', 'schooling', 'gni', 'internet']]
    prediction = model.predict([np.array(features)])
    output = round(float(prediction[0]), 2)

    if output < 0.40: category = 'Low HDI'
    elif output < 0.70: category = 'Medium HDI'
    elif output < 0.80: category = 'High HDI'
    else: category = 'Very High HDI'

    predictions.insert_one({
        'timestamp': datetime.utcnow(),
        'user_name': session.get('user'),
        'user_email': session.get('email'),
        'score': output,
        'category': category
    })
    return render_template('result.html', score=output, category=category)
```

## Activity 3.2: Implement MongoDB Integration for Authentication

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        users.insert_one({
            'name': request.form['name'],
            'email': request.form['email'],
            'password': request.form['password'],
            'role': request.form['role'],
            'created_at': datetime.utcnow()
        })
        return redirect('/login')
    return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = users.find_one({
            'email': request.form['email'],
            'password': request.form['password']
        })
        if user:
            session['user'] = user['name']
            session['email'] = user['email']
            return redirect('/')
    return render_template('login.html')

@app.route('/admin')
def admin():
    return render_template(
        'admin.html',
        total_users=users.count_documents({}),
        total_predictions=predictions.count_documents({}),
        active_models=2,
        logs=predictions.find().sort('timestamp', -1).limit(10)
    )

if __name__== '__main__':
    app.run(debug=True)
```

## Milestone 4: Frontend Development

Milestone 4 focuses on developing a cohesive, dark and gold-accented interface for HDI Insight across six pages: registration, login, home, prediction, result, and admin.

### Activity 4.1: Registration and Login Pages

- **register.html:** Collects Name, Email, Password, Confirm Password, and Role, submitting to /register.
- **login.html:** Collects the registered Email and Password, submitting to /login, with a link to register for new users.

### Activity 4.2: Home Landing Page

- Hero section: 'Estimate a Country's Development Index from four core indicators,' with a sample reading gauge and four headline statistics (R-squared score, input feature count, countries modeled, train/test split).
- 'What is HDI' section describing the four core indicators: Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users.
- Classification bands section listing the four HDI tiers, followed by a call-to-action linking to the Prediction Console.

### Activity 4.3: Prediction Console and Result Page

```
Country

Life Expectancy (Years)

Years of Schooling

GNI per Capita (USD)

Internet Users (%)

Predict HDI
```

result.html renders the predicted score and category on a horizontal gauge bar, with a 'Run Another Prediction' button returning the user to the Prediction Console.

### Activity 4.4: Admin Dashboard

admin.html displays summary cards for Total Registered Users, Total Predictions Made, and Active ML Models, followed by a Recent Predictions Log table listing date and time, user name, user email, prediction score, and category for each submission.

## Milestone 5: Deployment

In Milestone 5, the focus is on deploying HDI Insight locally, verifying MongoDB connectivity and correct end-to-end operation of authentication and prediction.

### Activity 5.1: Preparing the Application for Local Deployment

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt

# Ensure MongoDB is running
D:\projects>mongod
```

### Activity 5.2: Local Testing and Verification

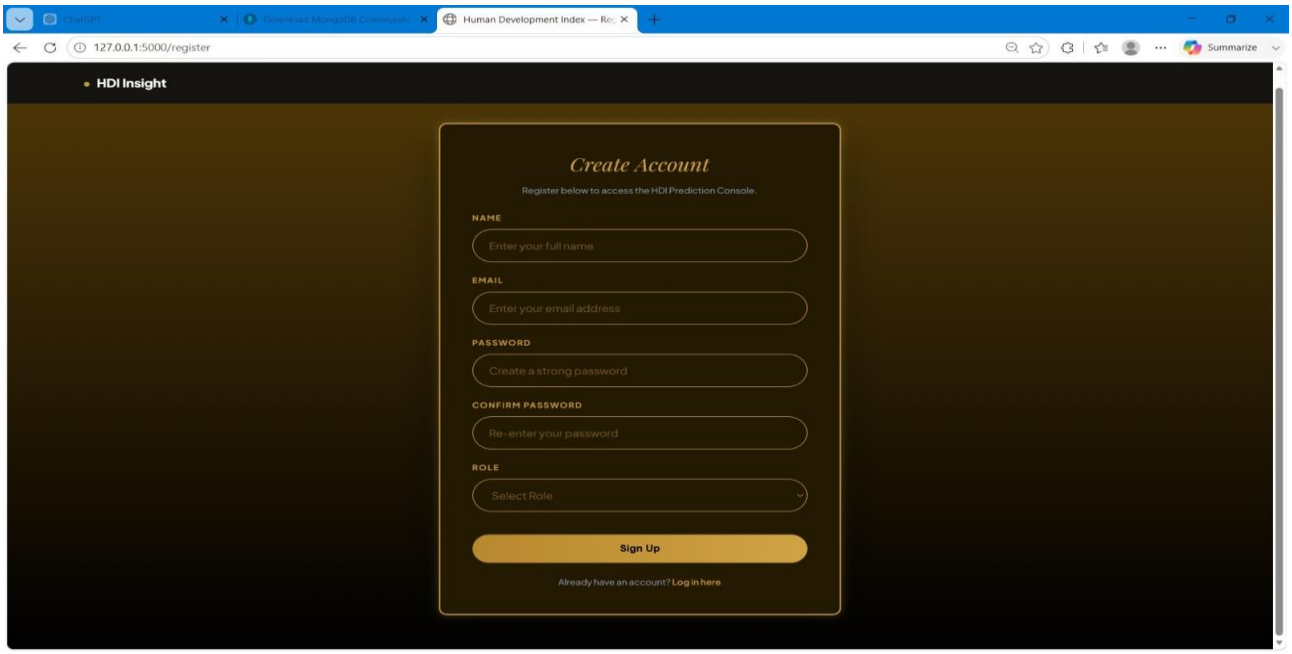
```
(myenv) D:\projects>python app.py

* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000
```

- Navigate to http://127.0.0.1:5000/register to create an account, then log in at /login.
- Review the home page statistics and HDI methodology, then open the Prediction Console.
- Submit indicator values for a chosen country within the documented ranges and click Predict HDI.
- Confirm the resulting score, gauge, and classification render correctly on the result page.
- Log in as an administrator and confirm the submission appears in the Admin Dashboard's Recent Predictions Log.

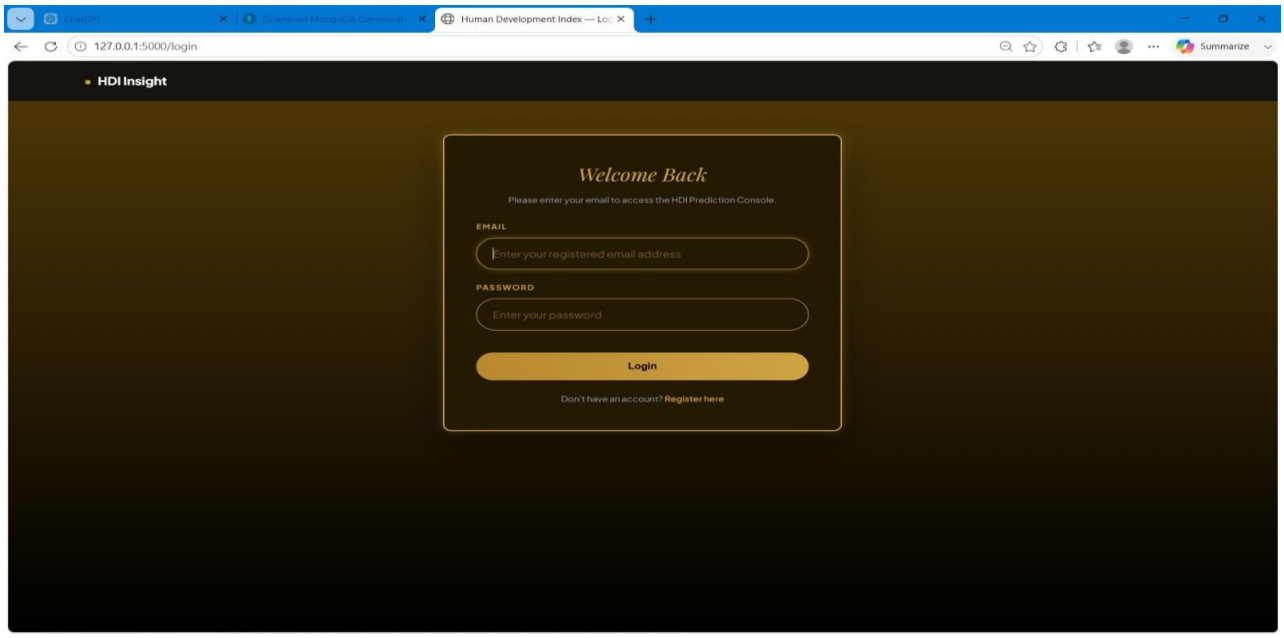
# Exploring the Website's Web Pages:

## Registration Page:



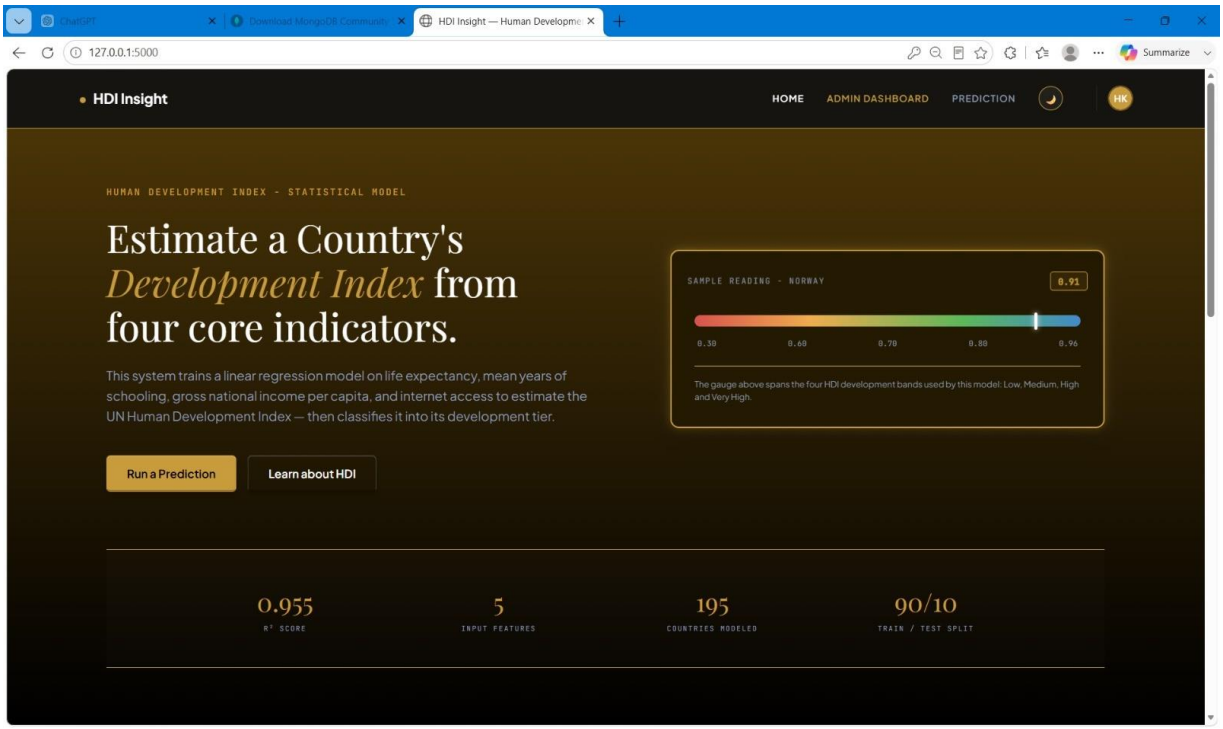

Description: The Create Account page collects Name, Email, Password, Confirm Password, and Role, submitting to MongoDB via the /register route.

## Login Page



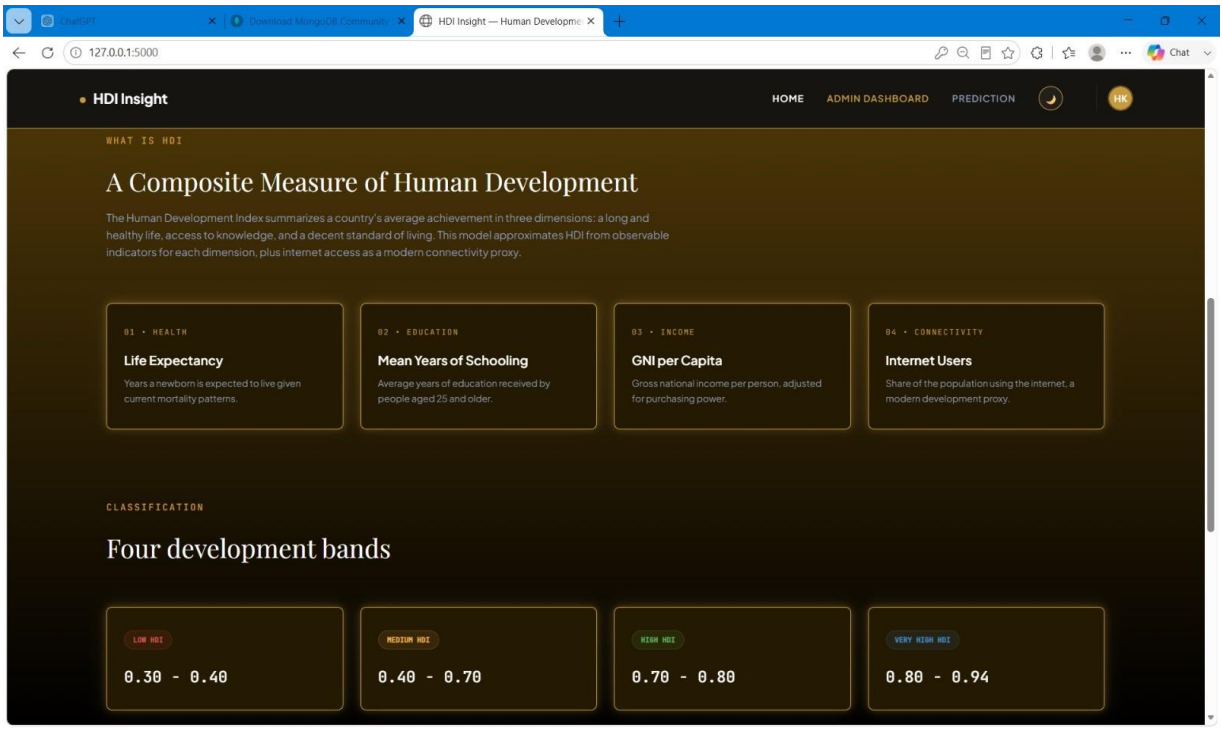

Description: The Welcome Back page authenticates returning users against the MongoDB users collection before granting access to the Prediction Console.

## Home Page — Hero Section:



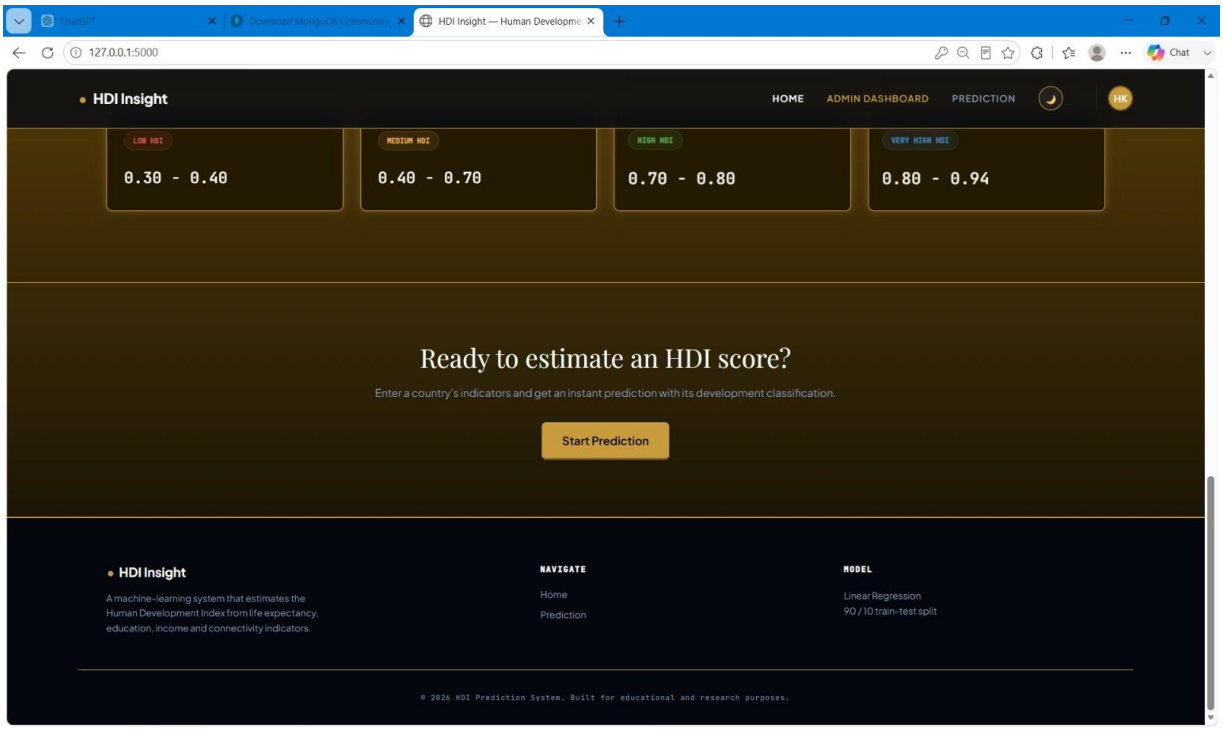

Description: The hero section introduces HDI Insight, showing a sample HDI reading and four headline statistics: R-squared score (0.955), input feature count (5), countries modeled (195), and the 90/10 train/test spli

## Home Page — What is HDI:



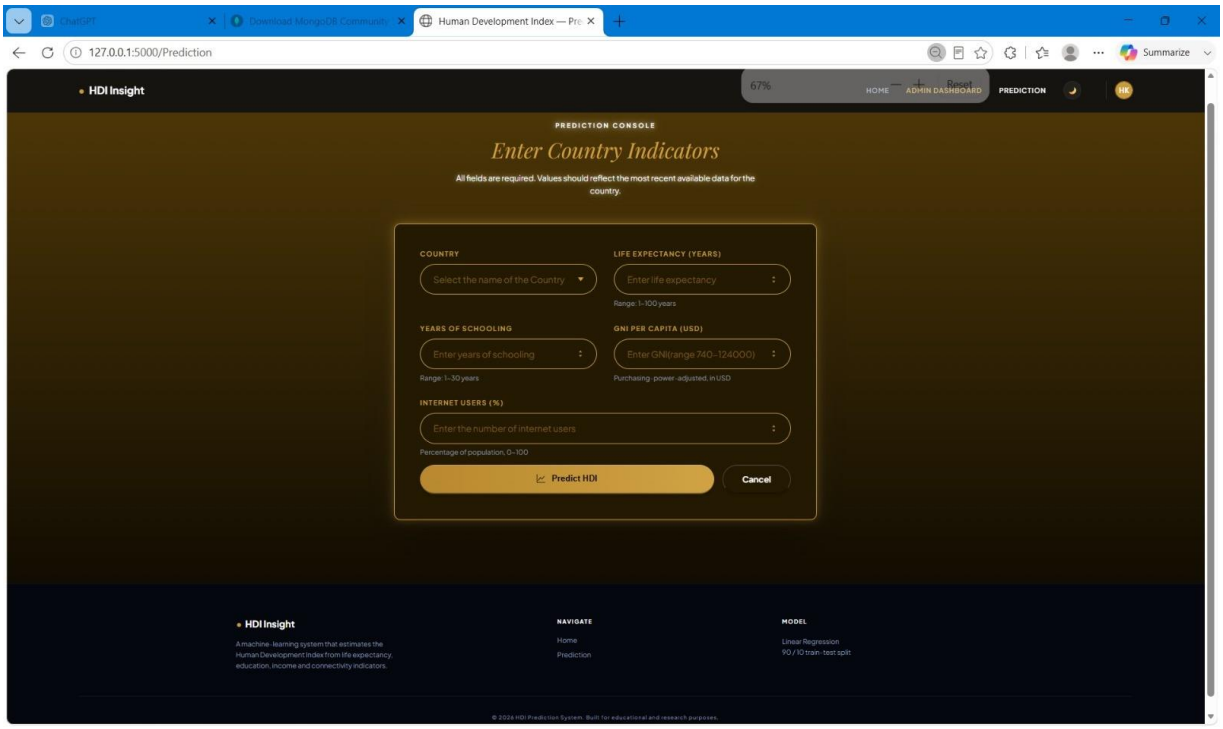

Description: This section explains the four core indicators used by the model — Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users — mapped to the Health, Education, Income, and Connectivity dimensions.

### Home Page — Classification Bands and Call-to-Action:



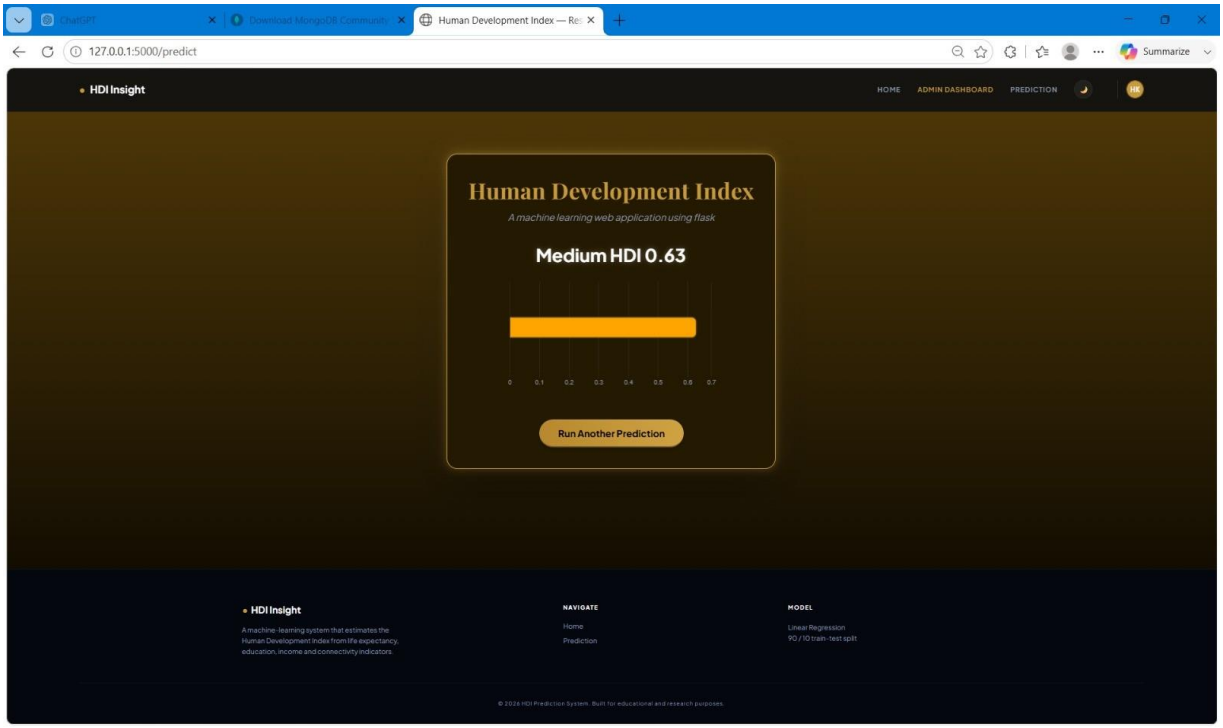

Description: The four HDI development bands (Low, Medium, High, Very High) are listed alongside a call-to-action inviting the user to start a prediction.

## Prediction Console



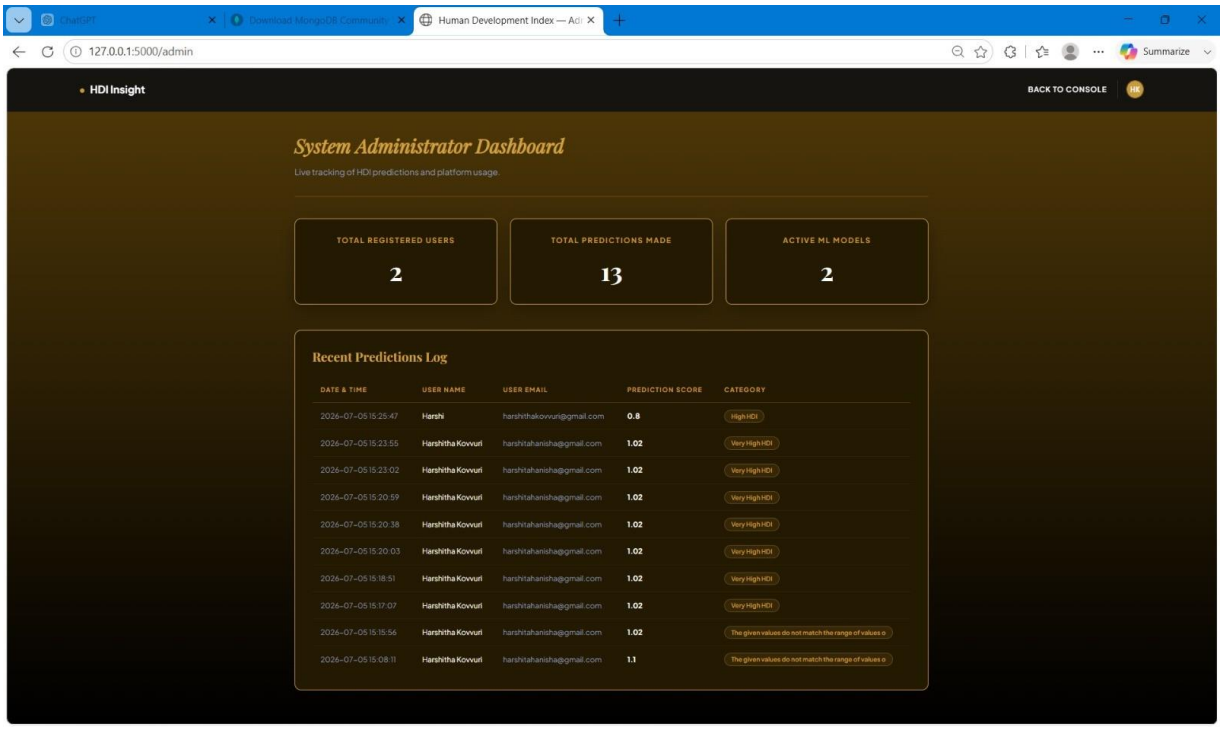

Description: The Prediction Console form collects Country, Life Expectancy, Years of Schooling, GNI per Capita, and Internet Users, each labeled with its valid input range, before submitting to the /predict route.

## Prediction Result




Description: The result page displays the predicted HDI score (Medium HDI, 0.63) on a horizontal gauge, with an option to run another prediction.

## Admin Dashboard




Description: The System Administrator Dashboard summarizes total registered users, total predictions made, and active ML models, followed by a Recent Predictions Log sourced from MongoDB.

## Conclusion:

HDI Insight is a machine learning platform built with Flask, scikit-learn, and MongoDB that streamlines the exploration of the Human Development Index. It uses a Linear Regression model trained on Kaggle's human development dataset to instantly estimate a country's HDI score from four key indicators — Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users — achieving an R-squared score of 0.955 on a 90/10 train-test split. The project, which included dataset selection, model training and evaluation, MongoDB-backed authentication, and deployment via Flask, highlights how a focused ML pipeline and a clean, authenticated web interface can make a UNDP-style development metric approachable for students, researchers, and policy enthusiasts alike. Looking ahead, HDI Insight offers opportunities for expansion, such as experimenting with additional regression algorithms, expanding the feature set from the full 82-column dataset, and extending the admin dashboard with usage trend charts. By continuously refining the model and frontend, the platform is well-positioned to evolve from an educational demo into a more comprehensive human development analytics tool.